

ESTIMATING THE TRAFFIC DENSITY FROM TRAFFIC CAMERAS

Vũ Lê Quỳnh Phương¹, Bùi Nhật Tài², Nguyễn Khắc Huy², Trần Nguyễn Minh Thu²,
Phạm Nguyễn Khang²

¹Kien Giang Teachers Training College, Kien Giang, Viet Nam

²College of Information & Communication Technology, Can Tho University, Can Tho,
Vietnam

vlqphuong@cds pkg.edu.vn; nhattaia6@gmail.com ;
huyb1606983@student.ctu.edu.vn; tnmthu@ctu.edu.vn;
pnkhang@ctu.edu.vn

Abstract

Traffic congestion is a daily occurrence in most urban of Vietnam, because of many reasons: an increasing number of vehicles, the rush hours (time to go to school, to work, after school, after work). To partly avoid these situations, it is necessary for a support system to provides information of traffic flow on key roads with traffic cameras at junctions, crossroads, in front of hospital gates, markets ... Estimating the traffic density from the traffic cameras helps the functional forces divide and divert traffic in time. In this research, the Faster R-CNN and YOLOv4 models is used to detect vehicles in traffic on the road, thereby estimating the density and sending alerts to the user. Experimental data were taken in front of Kien Giang hospital's gates, including 1260 images for training, 360 images for validation, and 3 sets of images for testing. The first set of 180 images has the same camera angle as the training set. The second set and the third set each contains 100 images with different camera angles and different camera angles from the training set. The results are about 92% of accuracy in detection of vehicles for the first set (2 models); 75% and 35% for the second set; 85% and above 20% for the third set and above 20% for the third set.

Keywords. Deep Learning; Convolutional Neural Network; Object Detection; Vehicle counting.

1 Introduction

In modern life, traffic jams in Vietnam are very common, especially, the more developed a city is, the more traffic jams occur. There are many surveillance cameras in the streets in Viet Nam, their purposes are to support crime prevention, traffic monitoring, however, the main of video surveillance is traffic management. Traffic management has become a practical need in modern life. When the traffic during peak

hours is managed, much useful information is monitored such as traffic density, traffic flow, lane occupancy can be obtained. Traffic density estimation is one of the key elements for traffic management, infrastructure, transportation planning and policy, pollution reduction, and traffic forecasting [11] Because of the lack of people, the police often sits on monitors to control traffic with their eyes and mind on the monitor during peak hours. With the development of computer vision as machine learning or deep learning, traffic surveillance videos are analyzed automatically. Over the world, there may be many kinds of research about estimating traffic density by using radar systems, loops, infrared, microwaves sensors. These existing methods are expensive and difficult to be implementation. In recent years, computer vision techniques have been applied in transport systems. Scientists use machine learning techniques, especially deep learning ones, to manage traffic and get positive results. However, the feature vehicles in Viet Nam are different because there are many motorbikes, bikes on the streets. There are no clear lanes in some streets. There are many purposes to set cameras in Viet Nam such as security, traffic monitoring,... so the installation location of cameras is not the same in streets. So that, we need an approach on this local traffic.

In this study, we propose an approach for estimating the traffic density from the traffic cameras in Kien Giang town (Viet Nam). The input data was taken in front of the Kien Giang hospital's gate. Then, vehicles are detected and counted. We have three labels: the vehicle has two wheels including a bike or motorbike; four wheels include vans, cars, buses; the last label is priority vehicles. The system sends a message to the user when amount vehicles over the value required.

2 Related work

Estimating traffic using vehicle detection is not a new trend. However, there is rarely any method specifically designed for vehicles in Viet Nam: many motorbikes and bikes, no reserved lane for vehicles. Work such as [4][5][9][10] build model traffic estimation system with traffic cameras.

Author Julian Nubert et al. have researched to build a traffic estimation system using the convolution neural network applied to traffic in Singapore [5]. Data is collected directly from traffic cameras in Singapore. The team of authors took photos from all cameras for over a week and selected the three different situations with different lighting and density conditions. In which, various traffic density conditions were collected over many days in clear road conditions and not covered by trees or obstructed by other objects. In total, the authors have collected 4582 photos. Then the dataset is randomly divided into 90% for training, 10% for validation. They also specified 5 classifiers for the images: "Empty" with 0-8 cars (almost empty street), "Low" with 9-20 cars (only a few cars), "Medium" with <50 car (slightly filled street), "High" with <100 cars (filled street or blocked lane) and "Traffic Jam" with > 100 cars (traffic almost not moving). The authors use the CNN model instead of the Recurrent Neural Network (RNNs) because RNNs will process continuous, ordered data; while the data collected for this study was done in such a way that per image

was extracted every 20 seconds. After the data is trained and optimized by some methods applying "transfer learning" on Inception V3, "class imbalance" and "masking", the results obtained the model with an accuracy of 74.3%. In this problem, the author has specifically applied for the traffic situation in Singapore. However, to approach and apply to the traffic situation in Vietnam with more diversified means of transport, mostly motorcycles, it is necessary to have an improved method or another more optimized model.

Author Nguyen Chien Thang has built a traffic counting system on the road using a combination of Object Recognition and Object Tracking [9]. The author used MobileNet SSD to recognize the object and then do the object tracking with the MOSSE Tracker. First, the author assumes a horizontal line (laser line), then iterates tracked objects to update them. During the loop, if an object passes the "laser line", count 1 new vehicle and unfollows that object, otherwise keep tracking the object. Every 5 frames, the author identifies the object once; if a new object appears, the system will follow up. Then it checks repeated the tracked subjects have exceeded the "laser line". In this article, the author used the MobileNet SSD trained model. With the test data, the traffic video is about 5 seconds long. However, in this article, the author focuses mainly on car identification (4 wheels). Therefore, to apply to traffic characteristics in Vietnam, there is necessary to have another topic dealing with the identification of more types of vehicles (motorbikes, bicycles).

In another study, Donato Impedovo et al [4], have built a traffic density prediction model based on object detection and density prediction. In the first phase, the algorithms: Haar Cascade, You Only Look Once (YOLO), Single Shot MultiBox Detector (SSD), and Mask R-CNN have been experimentally detected and compared the results with each other to choose the algorithm. The second stage, traffic density is predicted. Data used for the first stage (object discovery) includes 03 videos. The first is "M-30", the video was taken consists of 7520 frames with a resolution of 640x480 (30fps) on a sunny day. The second is "M-30-HD", the video captured consists of 9,990 frames with HD resolution 1280x720 (30fps) but into a cloudy day scene. The last video is "Urban1" consisting of 23435 frames with low-resolution 480x320 (25 fps). They did a "detect" frame by frame, then recorded the results of the traffic detection accuracy and execution time. The work is done on a computer with Ubuntu 18.04 operating system, equipped with AMD Ryzhen thread ripper 1920x 12 cores, Nvidia Titan RTX 24GB RAM, 64 GB DDR4 RAM. For the Mask R-CNN algorithm, the result obtained on data set "M-30" has an accuracy of 89%. The episode "M-30-HD" obtained better results with 91% of accuracy. But for the episode "Urban1", the result is only 41% of accuracy. Execution time is both 2.4 seconds - 3.0 seconds per frame. As such, the accuracy also depends partly on the quality of the images. Despite the high accuracy, the execution time for a frame is quite large from 2.4 seconds to 3.0 seconds. This system may have difficulty for real-time detection. With the data set taken at Urban are mainly 4-wheeled vehicles when applying in Vietnam with various types of vehicles, a more suitable topic is necessary.

Despite of the advances technologies, it is still challenging for vehicle estimating in countries that have many kind of vehicles. We conquer these challenges by a

system based on Faster-RCNN. Faster -RCNN has shown excellent performance for object detection on PASCAL VOC 2012.

3 Proposed Method

We use video dataset directly from the traffic camera to give the number of vehicles. According different place camera, we set different parameter to estimate density traffic. The system detects vehicles using a Faster-RCNN network. After that, we use the results from the model based on YOLOv4 (You Only Look Once version 4) to compare and evaluate.

The approach to traffic speed estimation in this challenge is outlined in Figure 1. Details in different phases are discussed as follows. First, the image is cut from the video every 5 frames. We use LabelImg to label the resulting images and convert them to a model-appropriate format using Roboflow. The image after labeling will be shuffled and divided into training and evaluation sets. The training data is used to train the model. During the training process, the evaluation set is used to evaluate and find the optimal parameters for the model. If the accuracy after the evaluation reaches over 90%, it will stop. Otherwise, we keep changing the parameters and continue training, evaluation. After obtaining the best evaluated trained model, we use that trained model to experiment on new videos. When adding a new video, the image is cropped every 20 seconds. It is preprocessed into the input normal form of the model and trained to predict the vehicles in the image with the location of that vehicle. Based on the location of the vehicles, determine the number of vehicles currently in the predefined vehicle count area. SMS notification will be sent based on the highest average result of 5 concluding attempts. The results will be saved to the database.

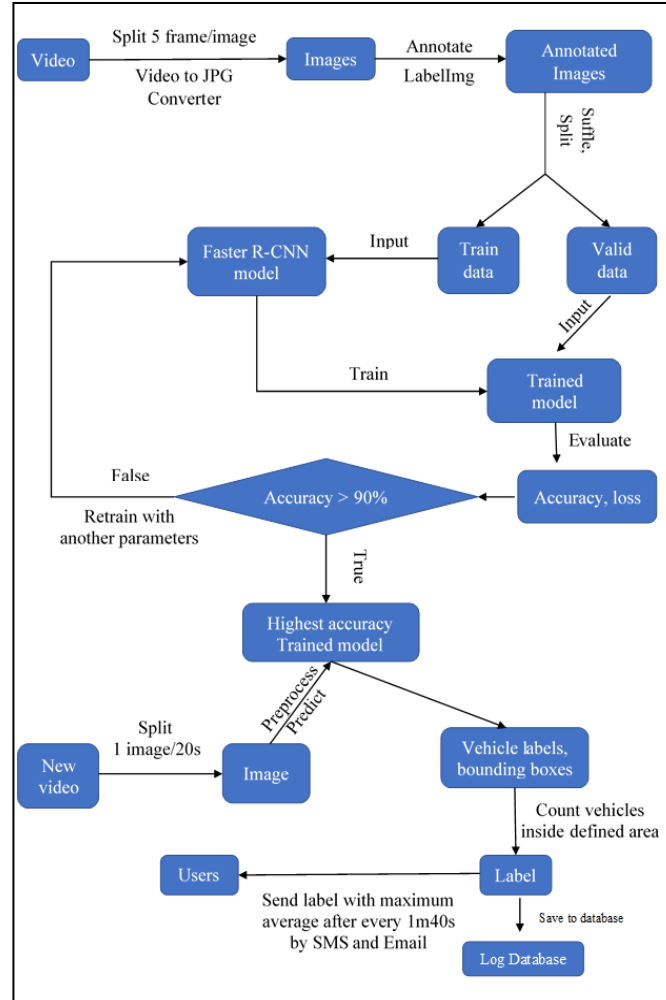


Fig. 1. The architerture of the proposed system

3.1 Faster RCNN structure

Faster R-CNN is an algorithm for finding the position of objects in an image. This algorithm will output boxes with the label .

Shaoqing Ren published Faster R-CNN, focusing on solving the key is the Selective Search algorithm from the weaknesses of R-CNN [10]. The author has replaced Selective Search with a CNN network called Region Proposal Network, the task of this network is to learn and indicate where the image area should be selected to search for objects and what percentage have objects in that area.

The processing flow of Faster-RCNN can be summarized as figure 2:

- Input image is passed through a CNN backbone, feature map is obtained. In this paper we use VGG16 backbone.
- An RPN subnet is used to extract regions called RoI (Region of Interest), or potentially feature-bearing regions from an image's feature map. Input of RPN is feature map, output of RPN consists of 2 parts: binary object classification (to distinguish objects from background, regardless of what the object is) and bounding box regression (to determine the image area likely to contain objects) object), so the RPN includes two loss functions and can be trained separately from the whole model.
- Faster-RCNN defines an additional concept of anchors (pre-defined boxes) that have been defined before training the model with purpose: the system doesn't directly predict the offset coordinates of the bounding box, it predicts indirectly through anchors.
- After going through the RPN, we will get the RoI region with different sizes (W & H). So that, we use RoI Pooling (available from Fast-RCNN) to get the same fixed size
- Finally, we split into 2 branches at the end of model, 1 for object classification with $N + 1$ (N is total number of classes, $+1$ is background) with bounding box regression. So we will define 2 loss components. The loss function of Faster-RCNN will include 4 components: 2 losses of RPN, 2 losses of Fast-RCNN.

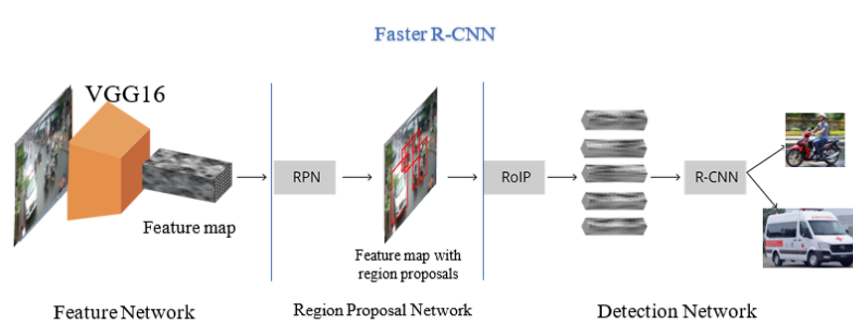


Fig. 2. The architecture of Faster R-CNN

3.2 API send SMS – Nexmo

Nexmo (currently renamed Vonage) provides communication API to connect between applications. Nexmo provides many APIs such as sending SMS, Voice, Video, Verify... Nexmo supports many languages such as: Python, Java, NodeJS, PHP, Ruby, ... Newbie programmers can easily access and use the most. Nexmo has a free trial policy that only sends SMS to pre-registered phone numbers. If you paid, SMS will be sent to all phone numbers as usual.

3.3 Emailing API - SendGrid

SendGrid is an API that provides email and other SMTP services. SendGrid also supports many languages such as: Python, Java, NodeJS, PHP, C #, Go, Ruby ... and support methods to send email Web API and SMTP Relay. SendGrid offers a free trial, sending and multiplying 100 emails per day for forever. There are also premium plans with full SendGrid functions and services.

3.4 Evaluation methods

After the model is trained. We use test dataset to evaluate model accuracy. This test dataset will not use "augmentation". Each image will be predicted the labels and "bounding boxes" in the image by the model. Then we compare the labels and the bounding boxes that were originally labeled to evaluate the model's accuracy.

Scenario of training, evaluation and testing for each parameter of the model with each test dataset is described as figure 3.

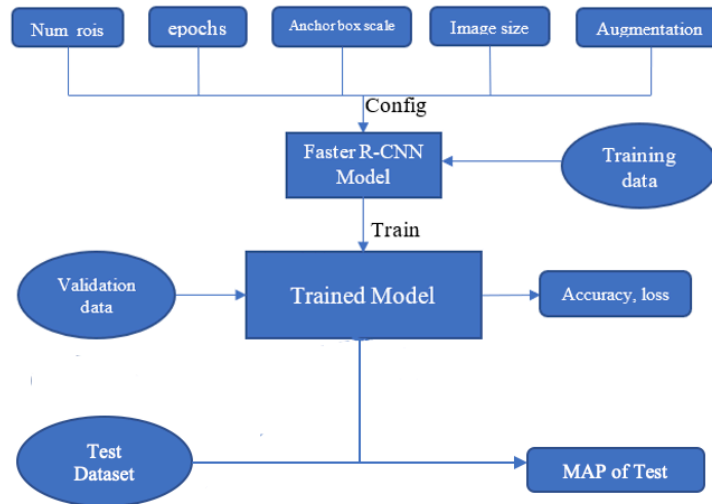


Fig. 3. Model evaluation checklist

4 Experiment results and discussion

4.1 The datasets

Dataset is provided by the police of Vinh Thanh Van ward - Rach Gia city - Kien Giang province. The cameras are located on Le Loi street (opposite the Kien Giang hospital). Each video is nearly 1 hour. The dataset has 03 camera angles (CAM1, CAM3, CAM5). From 10 AM to 11 AM, the scene was sunny (CAM 1, CAM 3); From 16:30 PM to 17:30PM, the scene was sunny first, but after that it started to rain (CAM 5). The resolution at the angle of CAM1 and CAM5 is 1920x1080, FPS is 15

frames / sec and 12 frames / sec, respectively. Particularly, the resolution at the angle of CAM3 is 1280x720, FPS: 10 frames / sec. The following images depict the camera angle of CAM1, CAM3, and CAM5.



Fig. 4. CAM1



Fig. 5. CAM3



Fig. 6a. CAM5 (sunny)



Fig. 6b. CAM5 (rainy)

Training, evaluation and testing dataset will be taken from the video in the CAM3 corner. Images were taken every 5 frames, then randomly partitioned the dataset into training, evaluation and testing datasets. We decided to define 3 labels include: bikes, cars , priority vehicles (pri).

Train dataset has 1260 images. Valid dataset includes 360 images. And 4 evaluation datasets: test dataset 1 (test1) includes 180 images from CAM3; test dataset 2 (test2) has 100 images from CAM1; test dataset 3 (test3) has 100 images from CAM5; Test dataset4 (Test4) has 100 images from CAM5 after these images are rotated 15 degrees.

4.2 Experiment results

The IoU ratio is used as a threshold for determining whether a predicted outcome is a true positive or a false positive. If the $\text{IoU} \geq \text{"threshold"}$ it is counted as "True Positive" (TP), otherwise it is "False Positive" (FP). There is also a property called "False negative" (FN), which is "ground truth", which was not detected. "Precision" is defined as the number of correct scores in the detected scores, calculated using the formula:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{TP}{\text{all detection}} \quad (1)$$

"Recall" is defined as a number of points true in the "ground truths":

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{TP}{\text{all ground truths}} \quad (2)$$

Suppose there are N values for calculating "precision" and "recall". Precision-Recall is a useful measure of success of prediction when the classes are very imbalanced. In information retrieval, precision is a measure of result relevancy, while recall is a measure of how many truly relevant results are returned. The Precision- Recall curve is drawn by drawing each point with the coordinates (P_n, R_n) ($n = 1, 2, \dots, N$) on the

coordinate axis and connecting them together. Average precision (AP) is determined by the formula:

$$AP = \sum_n (R_n - R_{n-1})P_n \quad (3)$$

In other words, the AP is the weighted sum of precisions at each threshold where the weight is the increase in recall.

To evaluate object detection models, the mAP is used. The mAP compares the ground-truth bounding box to the detected box and returns a score. The higher the score, the more accurate the model is in its detections.

$$mAP = \frac{1}{n} \sum_{k=0}^n AP_k \quad (4)$$

In equation (4), the AP_k value is the AP of class k ; n is the number of class.

Faster RCNN.

Through the results as table 1, when comparing the 1st and 2nd times, the larger image size is (300; 600), the longer it will take to process an image (1.5s; 2.8s). However, the accuracy when predicting a new image (mAP) will be higher (0.873; 0.911). But if the image size is too small (150), the possibility of missing important details is quite high. Specifically, in the 4th time, the execution time decreased insignificantly compared to the first time (1.1s > 1.5s), but the accuracy (mAP) decreased quite a lot (0.644 > 0.873).

The 2nd time and the third differ only in the parameter "num rois". It is found that when "num rois" is 512, the execution time will be faster than when "num rois" is 256 (2.6s > 2.8s) and the accuracy (mAP) is also higher (0.924 > 0.916) when the same camera angle as the training set (angle CAM3). However, when different camera angle compared to training video, the accuracy (mAP) is lower (0.563 > 0.572).

When the camera angle is different from the camera angle in training set (CAM1 > CAM3), the prediction accuracy (mAP) is generally lower. The highest test accuracy on the CAM1 angle is 0.75, while the highest on the CAM3 angle (camera angle in the training dataset) is 0.924.

For the parameter as in 4th time, it shows that identifying the motorcycles at the CAM5 angle is better. Different from the parameters of the other times, the test results on the same camera angle can identify the motorcycle better.

For the car label, it can be seen that the larger the parameter `img_size`, the higher the probability of correct prediction (2nd, 3rd), and vice versa, the smaller `img_size`, the lower correct prediction is (4th time).

The number of priority vehicles in the test sets is not enough, so the above results table 1 is only relative. In general, vehicle priority recognition is good for all parameters. The best is at the 4th time.

Con fig	Test1				Test 2			Test 3				Test 4			
	AP Bike	AP Car	AP Pri	mAP	AP Bike	AP Car	mAP	AP Bike	AP Car	AP Pri	mAP	AP Bike	AP Car	AP Pri	mAP
1	0.81	0.83	0.96	0.87	0.68	0.82	0.71	0.68	0.32	1	0.52	0.62	0.71	0.88	0.67
2A	0.89	0.93	0.95	0.92	0.65	0.66	0.68	0.65	0.62	1	0.65	0.68	0.76	1.00	0.72
2B	0.93	0.85	0.9	0.91	0.97	0.38	0.67	0.97	0.76	1	0.87	0.82	0.80	1.00	0.82
3A	0.88	0.87	0.93	0.9	0.69	0.53	0.55	0.69	0.61	0.33	0.64	0.68	0.80	0.35	0.73
3B	0.88	0.90	0.98	0.92	0.61	0.61	0.55	0.61	0.59	0.34	0.59	0.68	0.81	1.00	0.74
4	0.52	0.41	0.99	0.64	0.76	0.27	0.35	0.76	0.62	1	0.70	0.71	0.60	1.00	0.67
5A	0.84	0.80	0.98	0.87	0.74	0.75	0.70	0.74	0.73	0.33	0.71	0.64	0.62	0.21	0.62
5B	0.79	0.77	0.98	0.85	0.68	0.65	0.63	0.68	0.62	0.17	0.63	0.57	0.60	0.06	0.58
5C	0.76	0.79	0.90	0.82	0.72	0.78	0.75	0.72	0.70	1	0.72	0.60	0.64	1.00	0.62
5D	0.77	0.77	0.96	0.83	0.65	0.66	0.64	0.65	0.53	0.25	0.58	0.51	0.64	0.28	0.57
5E	0.74	0.74	0.97	0.82	0.68	0.69	0.67	0.68	0.60	0.33	0.62	0.60	0.62	0.29	0.60
5F	0.79	0.76	0.98	0.84	0.72	0.60	0.62	0.72	0.63	0.5	0.66	0.69	0.70	0.21	0.69
5G	0.73	0.80	0.88	0.80	0.59	0.58	0.62	0.59	0.63	0.2	0.59	0.58	0.75	0.76	0.67

Table 1. Detailed results with each data label on Faster- RCNN model

The chart in figure 7, we see that the result mAP of test 1 is the best. The results of test 2, test 3, test 4 are similar.

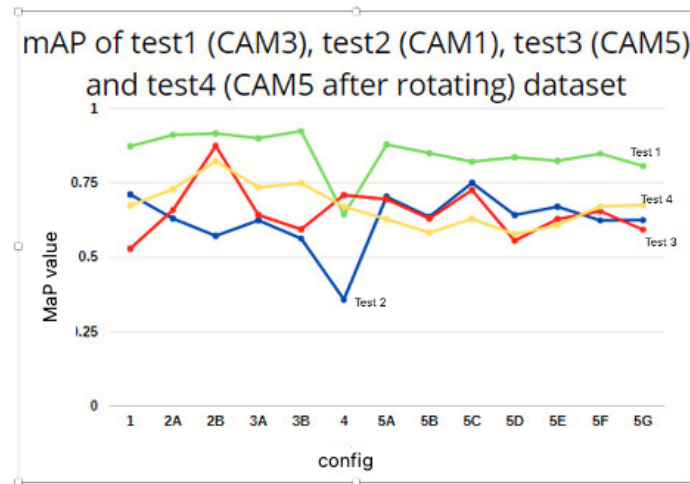


Fig. 7 The Results comparsion chart mAP of test 1, test2, test3, test4

Yolo.

You Only Look Once version 4 is developed by three developers Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao [2]. YOLO v4 is one of the best deep learning detection algorithm. Different from traditional models, YOLO investigates the image only once and detects it. YOLO v4 is the latest and the most advanced.

YOLOv4's architecture is composed of CSPDarknet53 as a backbone, spatial pyramid pooling additional module, PANet path-aggregation neck and YOLOv3 head. YOLOv4 is twice as fast as Efficient Det with comparable performance. In addition, AP (Average Precision) and FPS (Frames Per Second) increased by 10% and 12% compared to YOLOv3.

When the camera angle is the same from the camera angle in training set, the results show that the model has high accuracy at both thresholds 0.5 and 0.7.

Class Name	Ground truth	Detected	IoU ≥ 0.5			Detected	IoU ≥ 0.7		
			TP ₁	FP ₁	AP ₁		TP ₂	FP ₂	AP ₂
Pri	30	28	27	1	89.88%	28	27	1	89.88%
Bike	2222	2168	2059	109	92.12%	2168	2004	164	89.19%
Car	349	359	338	21	95.59%	359	336	23	94.53%
mAP of all classes:					92.53%				

Table 2. Detailed results with each data label in test dataset 1 on YOLOv4 model

When the camera angle is different from the camera angle in training set (CAM1 > CAM3), the prediction accuracy (mAP) is generally lower.

Class Name	Ground truth	Detected	IoU ≥ 0.5			Detected	IoU ≥ 0.7		
			TP ₁	FP ₁	AP ₁		TP ₂	FP ₂	AP ₂
Bike	601	635	450	185	68.32%	635	387	248	57.97%
Car	71	31	28	3	38.87%	31	27	4	37.58%
mAP of all classes:					53.59%				

Table 3. Detailed results with each data label in test dataset 2 on YOLOv4 model

The results showed that the model recognized bike (68.32%) better than car (38.87%), car were missing a lot (identified 31/71 vehicles) at 0 threshold.

When the camera angle and place is different from the camera angle in training set, the correct prediction rate of the model for the data that has not passed the camera angle with threshold = 0.5 is 23.25%, the reason is that the video is at an angle not correct with the learned data, so the Large objects are not detected. When threshold = 0.7, the rate drops sharply, especially in two-wheeled vehicles decreased from 48.7% to 10.52%.

Class Name	Ground truth	Detected	IoU ≥ 0.5			Detected	IoU ≥ 0.7		
			TP ₁	FP ₁	AP ₁		TP ₂	FP ₂	AP ₂
Pri	1	0	0	0	0%	0	0	0	0.00%
Bike	669	222	150	72	18.87%	222	107	115	10.52%
Car	82	55	45	10	50.88%	55	42	13	45.68%
mAP of all classes:					23.25%				18.73%

Table 4. Detailed results with each data label in test dataset 3 on YOLOv4 model

Program.

We use Pyqt5 library for interface design. Building a system with two main functions: traffic condition prediction based on video taken from traffic cameras and monitoring of traffic conditions that are recorded on the database.

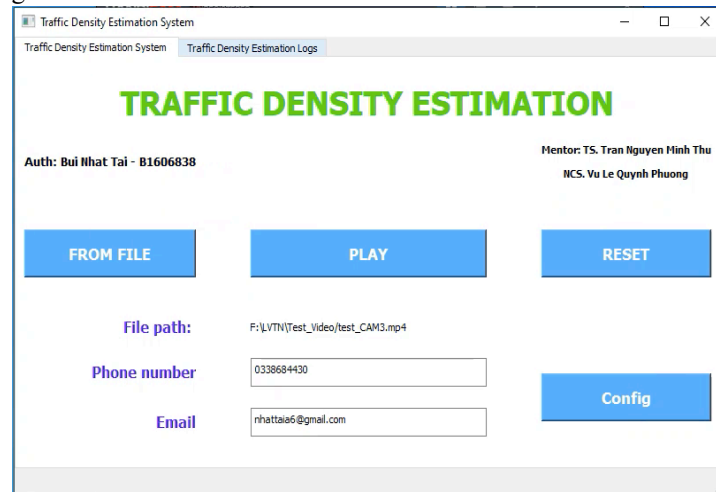


Fig. 8. Traffic density prediction function

The interface of traffic condition prediction function based on video is designed as figure 8. In this interface, we can choose file video to process. We fill in phone number and email to receive information.

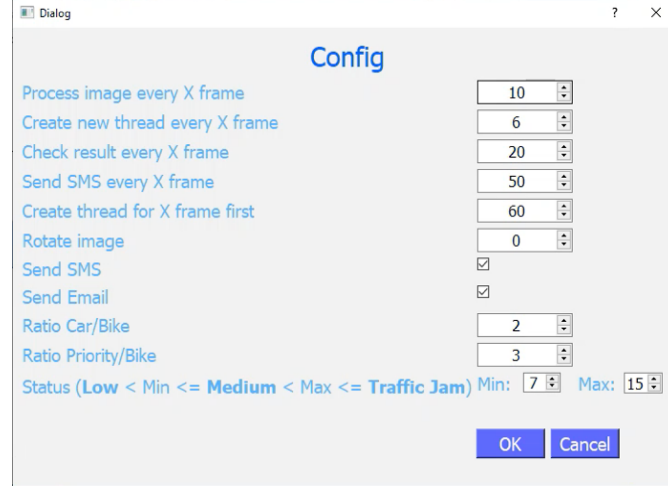


Fig. 9. Config interface

After the user enters all information and clicks the Config button, the config interface will be displayed as shown figure 9. In this dialog, we can choose the number of frame (X). It means after every X frame, the image is processed and the system give result to us. Besides, we can up or down the number of frame to create new thread after every that number. Sometimes, image received from the traffic camera is tilted. We can rotate image before the system predict to get high accuracy. The system can send email or SMS or not according “check”.

We can set the ratio of car and prioty vehicle to bike. Because the prioty vehicle need clear road and the car is bigger than the bike, so we counted the prioty vehicle as three or four bike. Similarly, we counted bikes as half or one third cars. The following formula is applied:

$$R = num_{bike} + (num_{car} * ratio_{car/bike}) + (num_{pri} * ratio_{pri/bike}) \quad (5)$$

Then, users set parameter Min, Max. The status have a result: Low, Medium or Traffic Jam. If R is less than MIN then the status is “LOW”. If R is grater than MAX then the status is “TRAFFIC JAM”. In the other case, the status is “MEDIUM”.

User select the parameters for the application and press play button. A picture frame will be displayed for the user to select the vehicle counting area. Users in turn left click to select the points that are the vertices of the vehicle counting area. The point of the vehicle counter will be green. When selecting more than 02 points, the application will automatically connect those points with the yellow line. Next, follow the parameters specified in the config step. In this example, the 50th frame of traffic status results will be sent to the user via SMS and email. For email, the text will be included with the photo. We illustrated in figure 10,11,12.

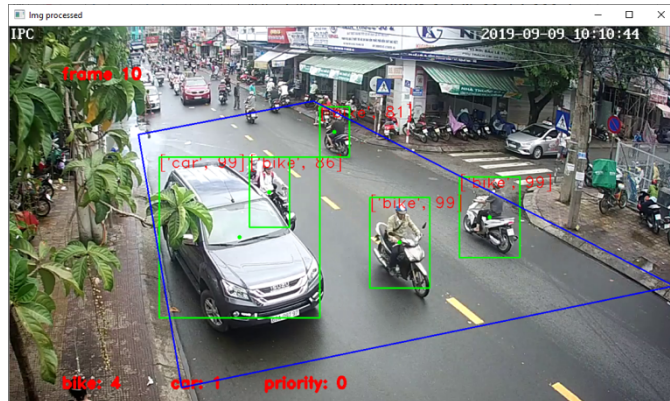


Fig. 10. The results of the vehicle count are in the 10th frame

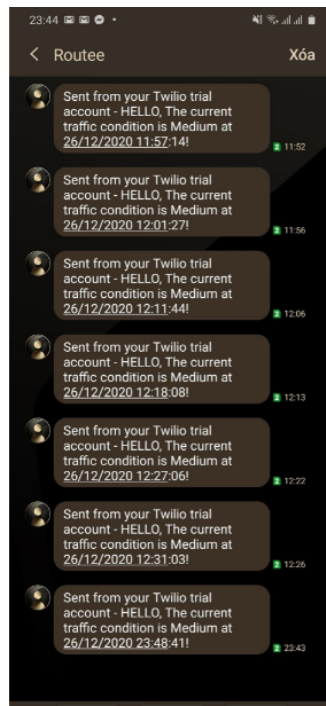


Fig. 11. SMS content of traffic status

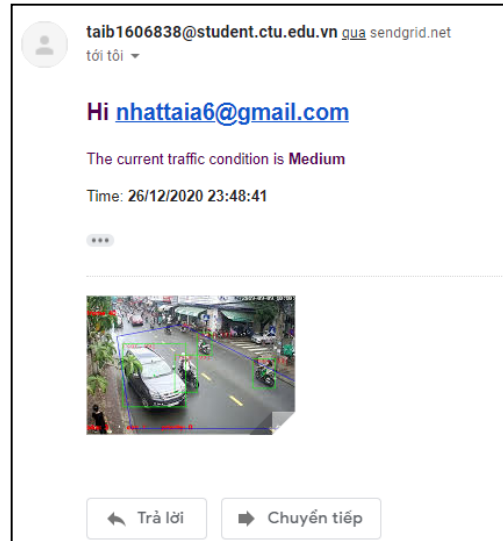


Fig. 12. Email content of traffic status

5 Conclusion and future work

We built a support system to track traffic status via camera traffic with relatively good average accuracy using the Faster R-CNN model (about 92%) and YOLOv4 (about 92.53%). We saw that the YOLOv4 model is better than Faster – RCNN when data train and data test are at the same angle. In another situation, YOLOv4 is worse than Faster-CNN although time to detect is shorter than the latter. The system applied on video 4 in sunny and rainy weather obtained 87% (Faster -RCNN) and 53.59% (YOLOv4) of accuracy although the camera angle of this video is different with training video (15 degrees).

However, when the vehicle is in heavy traffic (the vehicles overlap as shown in Figure 13), the model cannot identify correctly (cannot receive all vehicles in the area). During traffic jam, it is difficult to recognize vehicle because the image from video is an overlap vehicle. So, it is not feasible to count vehicles in the area when traffic jams occurs. Therefore, it is necessary to change the method of determining traffic density without relying on vehicle counting. It is possible to labels each image with low vehicle, average, crowded, traffic jam.

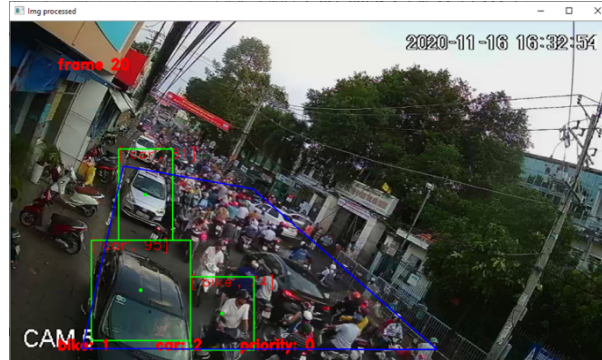


Fig. 13. Results when the road is full of traffic

Acknowledgment

The authors would like to thank Vinh Thanh Van ward police department in RachGia city (KienGiang, VietNam) for providing the traffic videos. We would also like to express our gratitude to Mr. Tu for his help in collecting and publishing the datasets.

6 References

1. Alan Lukezi, Tomas Vojir, Luka Cehovin Zajc, Jiri Matas and Matej Kristan: Discriminative Correlation Filter Tracker with Channel and Spatial Reliability, *International Journal of Computer Vision*, vol. 126, pp. 671-688 (2018)
2. Alexey Bochkovskiy, Chien-Yao Wang and Hong-Yuan Mark Liao: YOLOv4: Optimal Speed and Accuracy of Object Detection, *arXiv:2004.10934* (2020)
3. D. Zapletal, A. Herout, "Vehicle re-identification for automatic video traffic surveillance": *IEEE Conference on Computer Vision and Pattern Recognition Workshops, CVPRW*, pp. 1568-1574 (2016).
4. Donato Impedovo, F. Balducci, V. Dentamaro and G. Pirlo: Vehicular Traffic Congestion Classification by Visual Features and Deep Learning Approaches: A Comparison. *Sensor 2019* (2019).
5. Julian Nubert and Nicholas Giai Truong and Abel Lim and Herbert Ilhan Tanujaya and Leah Lim and Mai Anh Vu, Traffic Density Estimation using a Convolutional Neural Network, *arXiv:1809.01564* (2018).
6. K. Garg, S. Lam, T. Srikanthan, V. Agarwal: Real-time road traffic density estimation using block variance, *2016 IEEE Winter Conference on Applications of Computer Vision (WACV)*, page 1-9 (2016).
7. L. Zhuo, L. Jiang, Z. Zhu, J. Li, J. Zhang and H. Long: Vehicle classification for large-scale traffic surveillance videos using convolutional neural networks, *Machine Vision and Applications* 28, pp. 793-802 (2017).
8. Nal Kalchbrenner and Edward Grefenstette and Phil Blunsom, A Convolutional Neural Network for Modelling Sentences, *arXiv:1404.2188* (2014)

9. Nguyen Chien Thang: Kết hợp Object Detection và Object Tracking xây dựng hệ thống đếm phương tiện giao thông trên đường. <https://miai.vn/2020/01/13/ket-hop-object-detection-va-object-tracking-xay-dung-he-thong-dem-phuong-tien-giao-thong-tren-duong/> (2021)
10. Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun, Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks, arXiv:1506.01497 (2015)
11. Zhe Dai, Huansheng Song, Xuan Wang, Yong Fang, Xu Yun, Zhaoyang Zhang and Huaiyu Li: Video Based Vehicle Counting Framework, IEEE Open Access Journal, Vol.7, pp.64460-64470 (2019)